

강수현

엔진과 시스템의 원리를 이해하고, AI로 빠르게 탐색하며 직접 구현·검증하는 개발자

C++·Unreal Engine·Direct3D 11을 기반으로 엔진과 그래픽스의 동작 원리를 탐구하고, SkeletalMesh 런타임 처리 및 게임 시스템 최적화를 직접 구현했습니다.

문제를 구조적으로 분석하고 프로파일링을 통해 성능을 검증하며,
새로운 기술의 학습과 개발 과정에는 생성형 AI를 활용해 탐색과 구현 속도를 높이고 있습니다.

A 경력 · 애니펜

EXPERIENCE · UNITY

2023.1 - 2025.2 · 클라이언트 프로그래머 · 정규직 · 2년 2개월

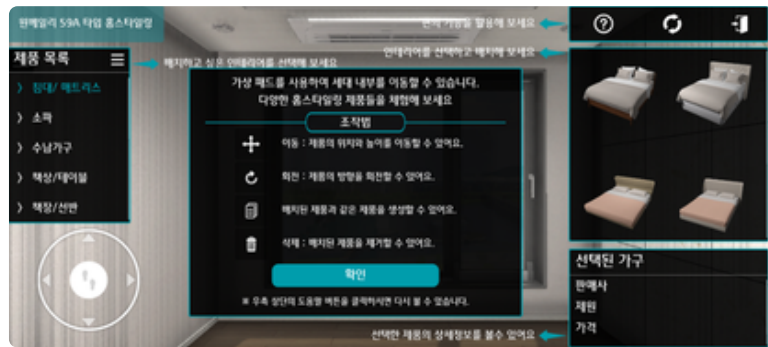
EXP.01 삼성물산 주거 플랫폼 '홈닉'의 메타버스 개발 iOS · Android 상용 출시

삼성물산 주거 플랫폼 '홈닉' 이용자 전용 연동 앱으로, 입주민이 자신의 아파트 단지를 3D 공간으로 둘러보고 상호작용하는 서비스 앱

2023.08 - 2023.11 · 4개월

초기개발

메타버스 서비스 앱



담당 R&R

- Addressables 기반 원격 에셋 사전 다운로드 시스템 구축 및 런타임 메모리 관리 구조 설계
- OIDC 통합 인증의 iOS 네이티브 브릿지 구현
- Command 패턴 기반 사내 UI 프레임워크 위에서 화면·팝업 흐름 구현
- iOS·Android 플랫폼별 빌드 이슈 및 크래시 대응

Unity <> C# iOS / Android Addressables <> Objective-C OIDC / OAuth2 UniTask



EXP.02 모푸샌드 IP 팝업스토어 서비스 앱

다양한 이벤트 참여 및 쿠폰 수령 기능과 3D 홈 화면을 제공하는 팝업스토어 방문객용 앱

iOS · Android 상용 출시 현재 서비스 종료

2024.07 - 2024.12 · 6개월 초기개발 모바일 서비스 앱

담당 R&R

- Strategy 패턴 + Behavior Tree 기반 **AI NPC 행동 시스템 설계**, 행동 6종 모듈화
- AI NPC **경로 탐색 교착(livelock) 문제 해결**
- Addressables 기반 원격 에셋 사전 다운로드 시스템 구축 및 **런타임 메모리 관리 구조 설계**
- 모바일 해상도·화면비 대응 반응형 UI 레이아웃 설계·구현
- iOS·Android 빌드 파이프라인 정립 및 스토어 심사 대응, **양대 스토어 상용 출시**

Unity <> C# Behavior Tree NavMesh Addressables UniTask
iOS / Android

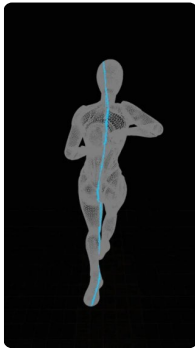
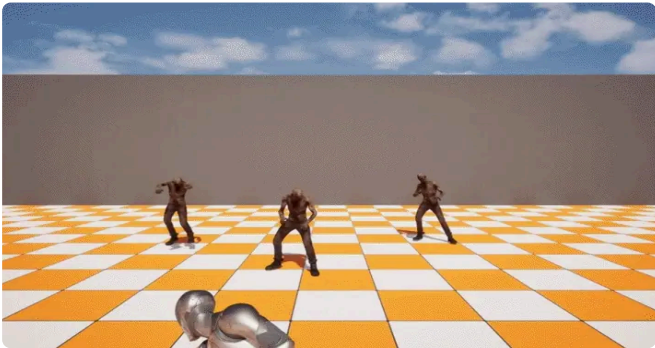
B 프로젝트 · UE5 / C++ PROJECTS

PRJ.01 Slice SkeletalMesh Plugin Fab 상용 출시

UE5 SkeletalMesh 실시간 절단 · 자동 절단면 생성 플러그인

2025.6 - 2025.8 · 2개월 3인 팀 크래프톤 정글 게임테크랩

Fab YouTube Docs Discord



담당 R&R

- SkeletalMesh **렌더 버퍼 런타임 재구성** 파이프라인 설계 및 구현
- 절단면 생성 및 CPU Skinning
- 절단면 Socket 부착 + VFX 통합
- YouTube 튜토리얼·데모 제작, Discord 커뮤니티 공지 및 사용자 커뮤니케이션

성과

Fab 상용 출시 후 운영

Docs(Notion) · Discord · YouTube 사용자 지원

절단 1회 총비용 17.87ms → 11.70ms (-34.5%)

렌더 스레드 대기 64회 → 0

정점 분류 6.50ms → 0.55ms (-91.6%)

렌더 버퍼 재구성 로직 8.04ms → 6.79ms (-15.5%, 격리 측정)

캡 웨이트 조회 0(N×M) → 0(1)

455/455 폴백 0회

PRJ.02 D3D11 Game Engine

Youtube ↗

Direct3D 11 자체 게임 엔진의 렌더링 파이프라인 설계·여러 피쳐 구현

2025.3 — 2025.6 · 3개월

주차별 4인 팀 재편성

크래프톤 정글 게임테크랩



담당 R&R

- RenderPass 생명주기 기반 4-Pass Deferred 파이프라인 구조 설계 (팀원 1인 협업) + PostProcess·Overlay·Depth 연결 단독 구현
- 공통 라이트 데이터 기반 4종 광원(Ambient·Directional·Point·Spot) 통합 + 조명 모델 4종 컴파일 타임 배리언트 Uber Shader 설계·구현
- Texture2DArray 기반 다중 Spotlight Shadow → slice DSV 직접 렌더링 · 3×3 PCF
- CPU 시뮬레이션 → GPU 인스턴싱 Sprite Particle, Viewer / Material UI 툴링

정량 성과

Deferred RT 메모리 -30.8%
Position RT 2장 제거 + Depth SRV 월드 위치 복원 · GBuffer+LightPass RT + Depth 기준

Shadow Depth 저장 2N → N
slice DSV 직접 렌더링으로 CopySubresource N → 0/frame

Particle 버퍼 생성 3N → 0/frame
Emitter마다 매 프레임 VB·IB·ParamBuffer 재생성 → 공유 StructuredBuffer 1회 생성으로 대체

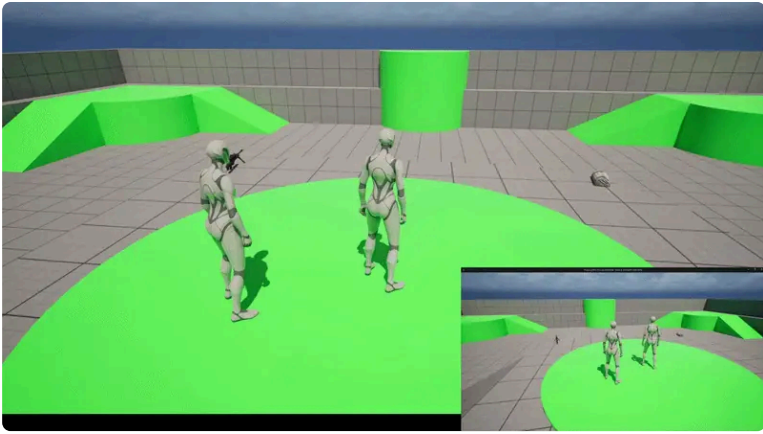
PRJ.03 project:EXFIL

GitHub ↗ Youtube ↗

UE5 데디케이티드 서버 환경에서 직접 설계한 Tarkov 스타일 인벤토리·전투 시스템 프로토타입

2026.3 — 2026.5 · 실 개발일 27일

1인 개발



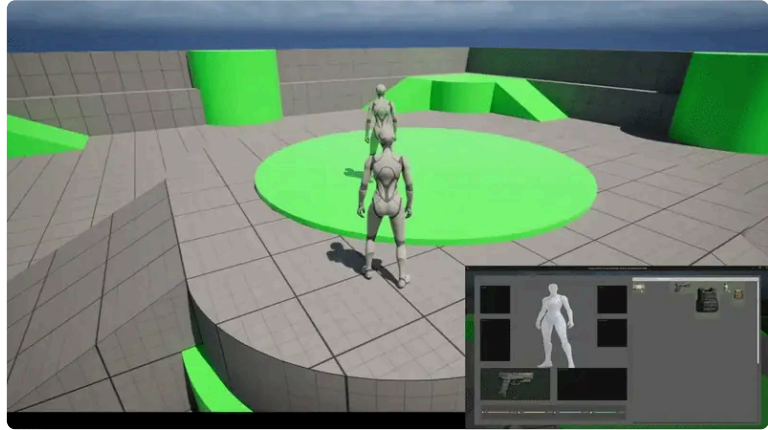
01 월드 아이템 획득 - 서버 권한 판정과 FastArray 델타 동기화



02 인벤토리 조작과 아이템 사용 - 비트마스크 배치 검증과 GAS 스탯 반영



03 아이템 제작 - 원자적 자료 차감과 취소 롤백



04 사격·사망·리스폰 - 서버 히트 재검증과 상태 기반 리스폰

담당 R&R

- 서버 권한(Server Authority) 아키텍처 설계
- 그리드 인벤토리 설계 및 **탐색 알고리즘 최적화**
- **FastArraySerializer** 델타 동기화 + **중분 캐시 패치** 구현
- 서버 히트 재검증(Lag Compensation) 파이프라인 구현
- MVVM UI 파이프라인 구축 및 GAS 통합

정량 성과

인벤토리 빈 슬롯 탐색 5.38배 개선 (31.06ms → 5.77ms)

후보 위치마다 아이템 크기만큼 셀을 다시 확인하던 방식을, 행 단위 비트마스크를 미리 합쳐두고 **후보 위치당 AND 연산 1회**로 교체

기준	10×20 그리드 / 50% 단편화 / 2×3 아이템 / 100,000회 반복
방법	고정 시드로 점유 패턴 재현, 위밍업 후 3회 측정 중 최단값 채택 (UE Automation Test)
범위	그리드 3종 × 점유 패턴 6종 × 아이템 5종 × 함수 2종 = 총 180개 측정값
결과	10×20 탐색 30개 케이스 평균 2.96배 / 최대 5.71배 개선, 90케이스 Before == After 검증

UE 5.6
 C++
 Dedicated Server
 GAS
 FastArray
 MVVM

C 기술 스택 TECH STACK

엔진 · 프레임워크
 Unreal Engine
 GAS
 Dedicated Server
 MVVM
 UMG / Slate
 Unity

언어	<> C++ <> C# <> HLSL / SM 5.0
그래픽스	Direct3D 11
네트워크	UE Replication RPC FastArray NetSerialization
형상 관리	Git / SourceTree Perforce
프로파일링 · 디버깅	RenderDoc Unreal Insights
개발 환경	<> Visual Studio 2022 <> Rider
협업 · 이슈 관리	Notion, Trello

D 연락처 & 링크	CONTACT
EMAIL	rkdtnigus3579@gmail.com
PHONE	010-4510-4260
GITHUB	github.com/agnesAqr ↗